

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)

Assessment Tool Weightage: 40%

QUESTIONS: 02

Total Marks:20

Annexure A

Experiments

Code of Conduct:

I **BANDARU JAYASRI-192424301** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Criteria	Conceptual Understanding	Accuracy of Answers	Application of Knowledge	Completeness of Response	Clarity & Presentation	Total
Weightage	30%	25%	15%	15%	15%	
Q1						
Q2						

Signature of the student

B.Jayasri

Total Marks

Annexure A

Short Answer Test

1. Design and conduct an experiment to implement a Naïve Bayes classifier on a real-world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.
2. Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability.
3. Implement the Expectation-Maximization (EM) algorithm for a mixture model and conduct experiments to study its convergence behavior. Vary initialization parameters and number of clusters, and observe their effect on convergence speed and final likelihood values. Record and analyze cases where the algorithm converges to local optima. Suggest strategies to improve convergence stability and performance.
4. Construct a Bayesian Belief Network for a given problem domain and perform inference under different evidence conditions. Conduct experiments by modifying conditional probabilities and observing changes in posterior probabilities. Analyze how dependencies between variables influence the results. Evaluate the robustness of the model and interpret how uncertainty is handled in probabilistic reasoning.
5. Implement a concept learning algorithm (such as Candidate Elimination) and perform experiments to analyze the effect of hypothesis space size on learning performance. Vary the number of attributes and training examples, and observe changes in sample complexity and convergence. Analyze how finite and infinite hypothesis spaces influence generalization ability and learning efficiency, and interpret results using the mistake bound model.
6. Design and conduct an experiment to implement a **Decision Tree classifier** on a real-world dataset. Train the model using different splitting criteria such as entropy and Gini index, and evaluate performance using accuracy and F1-score. Analyze the impact of tree depth and pruning on overfitting, and compare results across different configurations.
7. Perform an experiment using the **K-Nearest Neighbors (KNN)** algorithm on a classification dataset. Vary the value of K and distance metrics (Euclidean, Manhattan) to observe their effect on model performance. Evaluate using accuracy and confusion

matrix, and analyze how feature scaling influences the classification results and decision boundaries.

8. Implement a **Support Vector Machine (SVM)** model and conduct experiments using different kernel functions such as linear, polynomial, and RBF. Evaluate the model using accuracy and precision, and analyze how kernel choice and hyperparameters like C and gamma affect the decision boundary and classification performance.

9. Design an experiment to implement a **clustering algorithm such as K-Means** on an unlabeled dataset. Vary the number of clusters and initialization methods, and evaluate performance using metrics like inertia and silhouette score. Analyze how cluster separation and feature scaling impact the quality and stability of clustering results.

10. Conduct an experiment using a **logistic regression model** for binary classification. Train the model with and without regularization (L1 and L2), and evaluate performance using accuracy, precision, and recall. Analyze the effect of regularization on model complexity, overfitting, and interpretability of learned coefficients.

11. Design and conduct an experiment to implement a **Random Forest classifier** on a real-world dataset. Train the model with varying numbers of trees and maximum depth. Evaluate performance using accuracy, precision, recall, and F1-score. Analyze how ensemble size and feature randomness affect overfitting and generalization, and compare results with a single decision tree.

12. Perform an experiment using a **Principal Component Analysis (PCA)** technique for dimensionality reduction. Apply PCA on a high-dimensional dataset and vary the number of principal components retained. Evaluate the impact on model performance using accuracy and computational efficiency. Analyze how variance retention influences information loss and classification results.

13. Implement a **Gradient Descent optimization algorithm** and conduct experiments by varying learning rates and initialization values. Observe convergence behavior and record the number of iterations required to reach optimal values. Analyze cases of slow convergence or divergence, and suggest improvements such as momentum or adaptive learning rate methods.

14. Design an experiment to implement a **Neural Network classifier** for a classification task. Vary the number of hidden layers, neurons, and activation functions. Evaluate the model using accuracy and loss curves, and analyze how network architecture affects performance, convergence, and overfitting.

15. Conduct an experiment using a **hierarchical clustering algorithm** on an unlabeled dataset. Compare different linkage methods such as single, complete, and average linkage. Evaluate clustering quality using dendrogram analysis and silhouette score. Analyze how linkage criteria influence cluster formation and interpretability of results.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1) Aim

To implement a Naïve Bayes classifier on a real-world dataset, evaluate its performance using accuracy, precision and recall, and study the effect of feature independence and Laplace smoothing.

Dataset

The **Iris dataset** is used for this experiment. It contains 150 samples belonging to three classes:

- Setosa
- Versicolor
- Virginica

The features are:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

Algorithm

1. Load the Iris dataset.
2. Separate input features and target class.
3. Split the dataset into training and testing sets.
4. Train a Gaussian Naïve Bayes classifier.
5. Predict the classes of test samples.
6. Calculate accuracy, precision and recall.
7. Train the model using different feature sets.
8. Compare the results.
9. Analyze the effect of Laplace smoothing and probability estimation.

Python Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, precision_score, recall_score

# Load dataset
data = load_iris()
X = data.data
y = data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Naive Bayes model
model = GaussianNB()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Evaluation
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average="weighted")
recall = recall_score(y_test, y_pred, average="weighted")

print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall    :", recall)
```

Feature Set Experiment

The classifier can be trained with different feature combinations.

Feature Set	Observation
Sepal features only	Moderate performance
Petal features only	High performance
All four features	Best overall performance

Petal length and petal width provide strong information for distinguishing the Iris classes.

Laplace Smoothing

Naïve Bayes calculates probabilities from training data. If a particular feature value has zero probability for a class, it can cause problems during prediction.

Laplace smoothing avoids zero probabilities by adding a small value to frequency counts.

The formula is:

$$P(x|C) = \frac{\text{count}(x,C) + 1}{\text{count}(C) + k}$$

where k is the number of possible feature values.

For continuous Gaussian Naïve Bayes, smoothing is generally handled differently than in categorical Naïve Bayes. Therefore, the main experiment with Iris uses Gaussian probability estimation.

Effect of Feature Independence

Naïve Bayes assumes:

$$P(X|C) = P(x_1|C)P(x_2|C)\dots P(x_n|C)$$

This means the features are assumed to be conditionally independent given the class.

In the Iris dataset, some features are correlated, especially petal length and petal width. Therefore, the independence assumption is not completely true.

However, Naïve Bayes can still perform well because exact independence is not always required for good classification accuracy.

Probability Estimation

Gaussian Naïve Bayes estimates the mean and variance of every feature for each class.

The Gaussian probability density is:

$$P(x|C) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

where:

- (μ) = mean
- (σ^2) = variance

The class with the highest posterior probability is selected as the prediction.

Result

The Naïve Bayes classifier achieves high accuracy on the Iris dataset. Petal-based features generally provide stronger classification performance than sepal-only features.

Example result:

Metric	Result
Accuracy	~96–100%
Precision	~96–100%
Recall	~96–100%

The exact values can vary depending on the train-test split.

Analysis

The experiment demonstrates that:

1. Naïve Bayes is simple and computationally efficient.
2. Feature selection can significantly influence performance.
3. Correlated features can violate the independence assumption.
4. Despite this limitation, Naïve Bayes can achieve good classification performance.
5. Probability estimation determines the final class prediction.
6. Laplace smoothing is particularly important for categorical Naïve Bayes when zero-frequency problems occur.

Conclusion

Naïve Bayes provides an efficient probabilistic approach for classification. The experiment shows that appropriate feature selection improves classification performance. Although the conditional independence assumption is not always realistic, the classifier can still achieve good results. Laplace smoothing helps prevent zero-probability problems, while reliable probability estimation is essential for accurate classification.

2) Aim

To estimate the parameters of a probability distribution using **Maximum Likelihood Estimation (MLE)** and compare the results with Bayesian estimation for different sample sizes.

Concept

Suppose observations are assumed to follow a normal distribution:

$$[X \sim N(\mu, \sigma^2)]$$

The parameters to estimate are:

- Mean (μ)
- Variance (σ^2)

Maximum Likelihood Estimation

MLE chooses the parameters that maximize the likelihood of observing the given data.

For a normal distribution:

$$[\hat{\mu}_{MLE} = \frac{1}{n} \sum_{i=1}^n x_i]$$

and

$$[\hat{\sigma}^2_{MLE} = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{\mu})^2]$$

Bayesian Estimation

Bayesian estimation combines:

- Prior knowledge
- Observed data

using Bayes' theorem:

$$P(\theta|D)$$

$$\frac{P(D|\theta)P(\theta)}{P(D)}$$

where:

- $P(\theta)$ = prior distribution
- $P(D|\theta)$ = likelihood
- $P(\theta|D)$ = posterior distribution

Dataset

Consider the following observations:

10, 12, 11, 13, 12, 14, 11, 13, 12, 10

These observations can be used to estimate the mean and variance.

Python Program

```
import numpy as np

# Dataset
data = np.array([10, 12, 11, 13, 12, 14, 11, 13, 12, 10])

# MLE estimation
mle_mean = np.mean(data)
mle_variance = np.var(data)

print("MLE Mean      :", mle_mean)
print("MLE Variance :", mle_variance)

# Bayesian estimation
prior_mean = 12
prior_variance = 4

n = len(data)
sample_mean = np.mean(data)

# Posterior mean for known variance
posterior_variance = 1 / (n / mle_variance + 1 / prior_variance)

posterior_mean = posterior_variance * (
    (n * sample_mean / mle_variance) +
    (prior_mean / prior_variance)
)

print("Bayesian Mean:", posterior_mean)
print("Bayesian Variance:", posterior_variance)
```

Experimental Comparison

The experiment can be conducted using small and large samples.

Small Sample

Suppose only:

10, 12, 11

are available.

With a small sample, MLE depends heavily on the observed values. The estimate may therefore be unstable.

Bayesian estimation incorporates prior knowledge, which can provide a more stable estimate.

Large Sample

Suppose many observations are available.

As the sample size increases, the effect of the prior becomes smaller.

Therefore:

[
Posterior $\approx$ Likelihood
]

and Bayesian estimation approaches the MLE estimate.

Comparison

Factor	MLE	Bayesian Estimation
Uses data	Yes	Yes
Uses prior knowledge	No	Yes
Small sample performance	Can be unstable	Usually more stable
Large sample performance	Reliable	Reliable
Computational complexity	Low	Higher
Output	Single parameter estimate	Posterior distribution/estimate
Prior assumptions	Not required	Required

Effect of Prior Assumptions

Suppose previous medical or business knowledge suggests that the expected mean is approximately 12.

A Bayesian model can represent this using:

```
[  
\mu \sim N(12,4)  
]
```

The observed data is then combined with this prior.

If the dataset is small, the prior can have a noticeable effect on the posterior estimate.

If the dataset is large, the observed data dominates the prior.

Sample Size Analysis

Small Sample

- High uncertainty
- MLE can vary significantly
- Bayesian prior can stabilize the estimate
- Prior assumptions have greater influence

Large Sample

- Lower uncertainty
- MLE becomes more reliable
- Bayesian and MLE estimates become closer
- Prior influence becomes relatively small

Example Interpretation

If the observed sample mean is close to the prior mean, both methods will produce similar estimates.

If the sample is small and differs significantly from the prior, Bayesian estimation produces a compromise between prior knowledge and observed data.

This can be represented as:

```
[  
Posterior =  
\text{Prior Information} +  
\text{Evidence from Data}  
]
```

Results

The experiment shows that:

1. MLE estimates parameters directly from observed data.
2. Bayesian estimation combines observed data with prior knowledge.
3. Small datasets are more sensitive to prior assumptions.
4. Large datasets reduce the influence of the prior.
5. Both methods produce increasingly similar estimates as sample size increases.

Conclusion

MLE is simple and effective when sufficient data is available. Bayesian estimation is useful when prior knowledge is available or when the dataset is small. The Bayesian approach provides a systematic way to combine previous knowledge with new observations. As sample size increases, the influence of the prior decreases and the Bayesian estimate approaches the MLE estimate.